

# Almost Segment Tree

# 今天讲啥？

|                                  |
|----------------------------------|
| 7月14日                            |
| 树状数组+线段树（基础线段树、权值线段树、动态开点、线段树合并） |
| 模拟题一套                            |

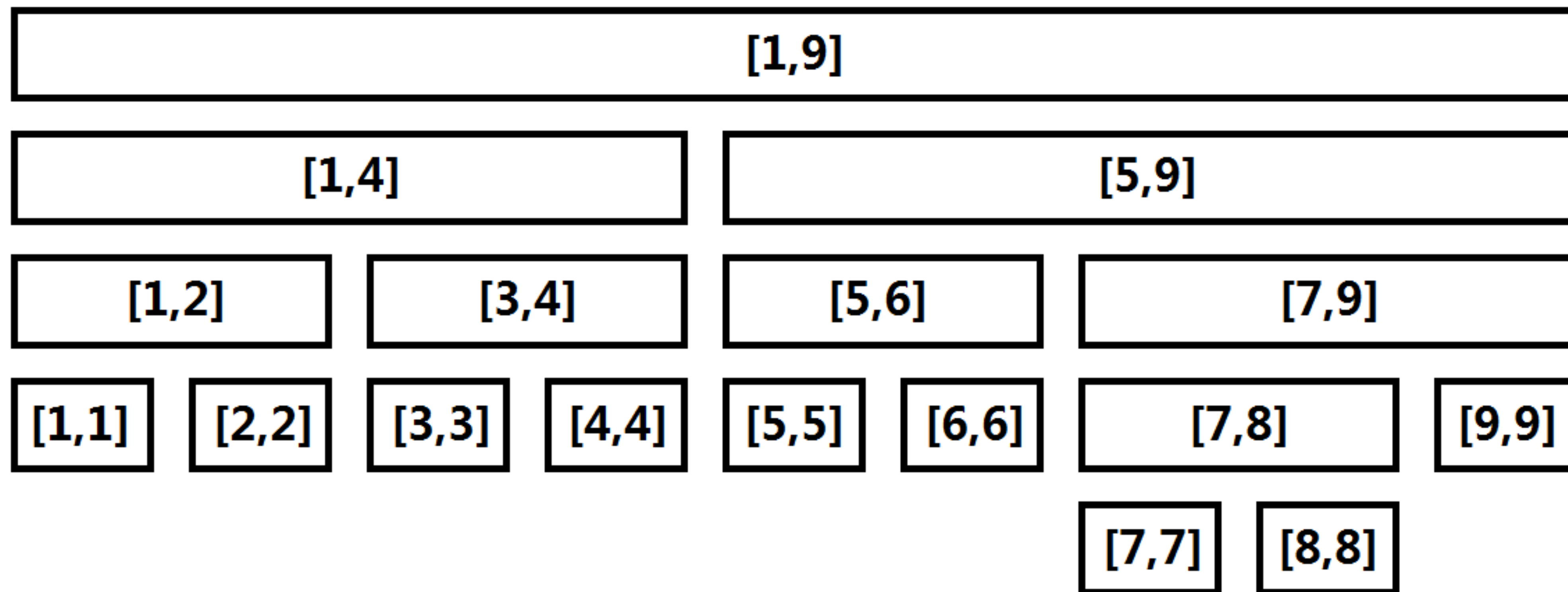
|        |    |                                                |                                         |                                                                                            |                          |
|--------|----|------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------|--------------------------|
| 高级算法1班 | 60 | 熟练掌握各类数据结构和算法，获得过提高组一等奖或具备同等水平，并准备参加次年省队选拔赛的同学 | 熟练掌握各类数据结构和算法，获得过CSP或者NOIP提高组一等奖或具备同等水平 | 树状数组，线段树，可持久化数据结构，树形DP、状压DP，动态规划优化，Tarjan算法及连通性问题，字符串问题选讲（Hash，kmp，trie，AC自动机）和数论进阶以及题目选讲等 | 机房讲专题课为主<br>上机训练和综合测试、题解 |
|--------|----|------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------|--------------------------|

# 树状数组/线段树

- ~~大家已经是「高级算法班」的选手了，可能不少人已经知道这些是啥了。~~
- 为了防止大家不知道，我们还是先简要介绍一下线段树与树状数组。
- 如果你会了可以就当这些简要介绍原理供大家复习。同时也会准备一些简单的小习题供你秒。



# 线段树



# 构建

|               |               |               |               |
|---------------|---------------|---------------|---------------|
| d[1]=60 [1,5] |               |               |               |
| d[2]=33 [1,3] |               | d[3]=27 [4,5] |               |
| d[4]=21 [1,2] | d[5]=12 [3,3] | d[6]=13 [4,4] | d[7]=14 [5,5] |
| d[8]=10 [1,1] | d[9]=11 [2,2] |               |               |

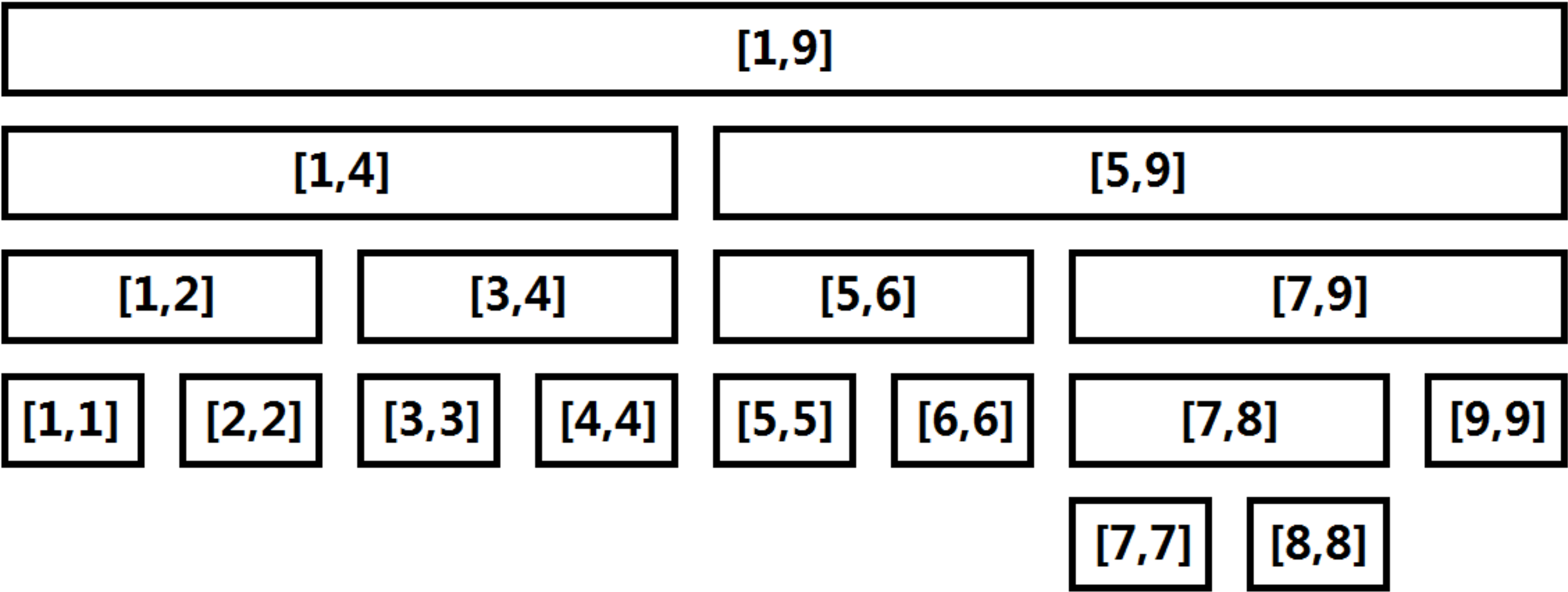
# 单点修改/查询

- 单点查询：沿着树上走到对应的点，查询对应的值。
- 单点修改：沿着树上走到对应的点，进行修改。
- 为了支持区间询问，我们会对每个点维护区间信息合并后的结果。
  - 例如如果是维护区间和，那么就维护这个节点子树内所有点的信息和。如果是区间最小值，就维护这个子树内所有点的最小值。



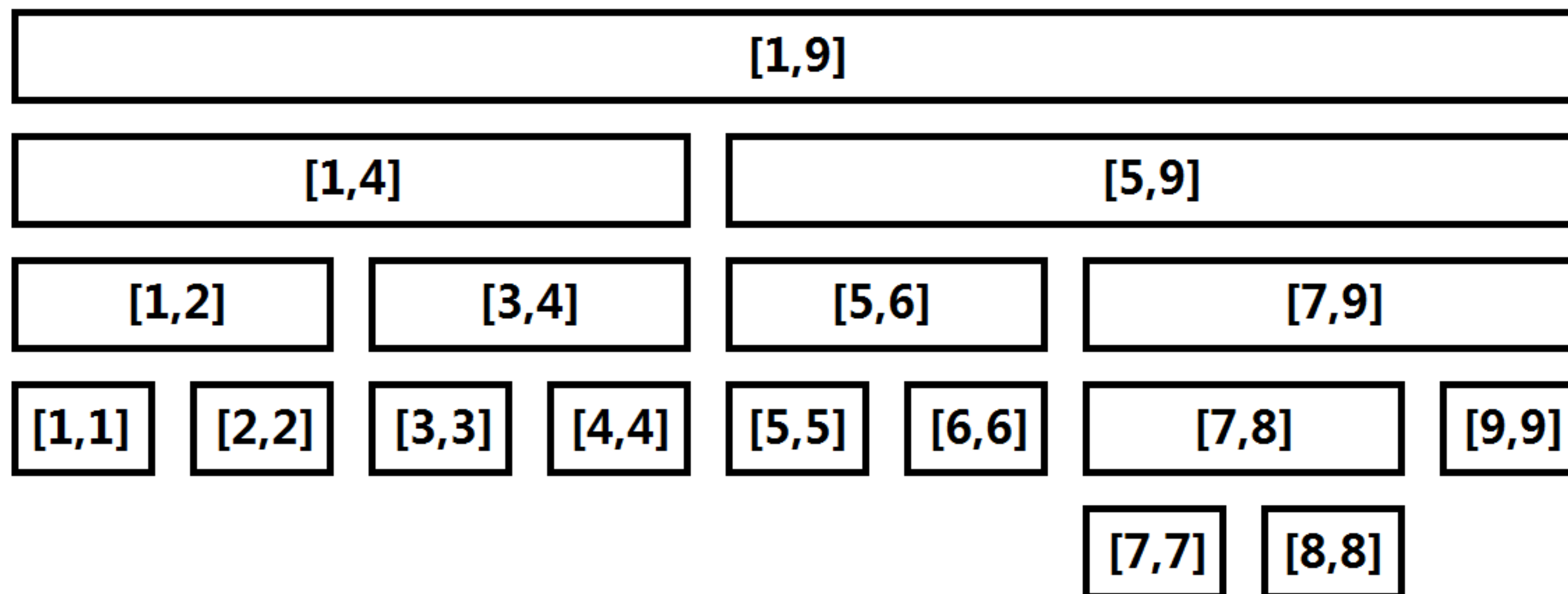
# 节点信息

- 单点加，区间求和



# 懒标记

- 区间加，区间求和





# 以区间求和为例

- “子树内还需要去加的值”
- 贡献
  - `maintain(u, l, r, v)`

# 本质

- 我们知道，线段树可以维护区间和、区间最值、区间矩阵乘积等信息。
- 同过一些手法，我们也可以拿来维护诸如最大子段和。
- 我们对信息所要求的性质有什么？以及怎样的标记可以被线段树维护？

# 信息的性质

- 什么是“可合并”？
- 以区间和、区间最值、区间矩阵乘积为例？
- 线段树的结构中按照划分出的区间来合并信息  $\Rightarrow$  信息本身要满足结合律
- 打标记时，标记需要可合并
- pushdown的顺序不做要求  $\Rightarrow$  标记的合并具有结合律



# 一组小例子

- 区间最大子段和  $\Rightarrow$  容易
- 单点修改区间最大子段和  $\Rightarrow$  容易
- 区间修改区间最大子段和?
  - 类比刚才的描述，他困难在哪里？
  - 标记对信息可否快速合并

# Furthermore...

- 接下来掏出一手 rqy 的博客 [https:// rqy.blog.uoj.ac/blog/8105](https://rqy.blog.uoj.ac/blog/8105)
- 不出意外，接下来几页不会讲。如果最后有空余时间会回过来做个简介。

# 信息的合并

## 标记和信息

众所周知——也或者没有那么周知，线段树通过打标记维护信息的时候，标记和信息分别要构成半群，并且之间的作用要具有分配律，此之谓“双半群”。具体地说，

- 线段树每个结点上有一个“标记”，其具有类型  $T$ ，或者说属于集合  $T$ 。
- 每个结点还有一个“信息”，其具有类型  $D$ ，或者说属于集合  $D$ 。
- 由于我们要合并信息，必须有一个合并函数  $\text{merge}(D, D) \rightarrow D$ 。如果我们维护的是区间和，那么合并就是加法。
- 如果当前结点有懒标记，那么为了得出这个结点的信息（也就是所谓的 **pushup** 或者 **update**），除了要合并两个子树的信息之外，还要考虑合并之后的信息在这个懒标记应用上去之后变成什么。这就要求我们有一个“作用”函数  $\text{apply}(T, D) \rightarrow D$ 。
- 打标记或者下传标记的时候，有可能要打标记的结点原来就有标记。这就需要我们知道两个标记先后作用之后会变成什么标记，也就是一个标记复合函数  $\text{compose}(T, T) \rightarrow T$ 。如果对一个区间先打上  $t_1$  标记，再打上  $t_2$  标记，结果应该和打上  $\text{compose}(t_2, t_1)$  标记相同。

举个例子。假设我们操作是区间乘和区间加，要维护区间和。那么：- 标记  $T$  就形如  $(a, b)$ ，表示要把区间里每个数  $x$  都变成  $ax + b$ 。- 信息  $D$  就形如  $(l, s)$ ，其中  $l$  表示区间长度， $s$  表示区间和。虽然其实这个  $l$  永远不会变所以不用维护，我们只是形式地这么写一下而已。- 信息的合并就是简单的  $\text{merge}((l_1, s_1), (l_2, s_2)) = (l_1 + l_2, s_1 + s_2)$ 。- 标记对信息的作用是  $\text{apply}((a, b), (l, s)) = (l, as + bl)$ 。- 标记的复合是  $\text{compose}((a_1, b_1), (a_2, b_2)) = (a_1a_2, a_1b_2 + b_1)$ 。注意我们的符号是先作用第二个再作用第一个。

如果我们只有区间乘，那标记就是一个数表示区间乘  $x$ ，信息就是一个和  $s$ 。合并就是加法，作用和复合都是乘法。因此，我们在一般情况下，也用加法符号  $+$  表示合并，乘法符号  $\times$  或者  $*$  表示作用和复合。

上面的例子就变成 -  $(l_1, s_1) + (l_2, s_2) = (l_1 + l_2, s_1 + s_2)$ 。-  $(a, b) * (l, s) = (l, as + bl)$ 。-  $(a_1, b_1) * (a_2, b_2) = (a_1a_2, a_1b_2 + b_1)$ 。



# “双半群”结构

接下来我们想一想标记和信息组成的这个结构，或者说系统，具体要满足哪些条件。

1. 首先，区间信息合并的先后顺序是无所谓的。如果有三个区间  $[l, x], [x + 1, y], [y + 1, r]$ , 他们对应的信息分别是  $d_1, d_2, d_3$ , 那么总应该有

$$\text{merge}(\text{merge}(d_1, d_2), d_3) = \text{merge}(d_1, \text{merge}(d_2, d_3)).$$

或者说  $(d_1 + d_2) + d_3 = d_1 + (d_2 + d_3)$ . 也就是说信息的合并满足结合律.

2. 标记也应该有结合律:  $t_1(t_2t_3) = (t_1t_2)t_3$ .
3. 而且，根据我们“作用  $\text{compose}(t_1, t_2)$  等价于先作用  $t_2$  再作用  $t_1$ ”的原则，总应该有  $(t_1t_2)d = t_1(t_2d)$ .

*这也就是为什么我们规定成先  $t_2$  后  $t_1$ ，不然写起来顺序就乱了.*

4. 此外，我们发现，标记下传之后，当前结点的信息是不会变的。也就是说，两个子树的信息先合并再应用标记，或者先应用标记再合并，得到的应该是一样的。这样就是说  $t(d_1 + d_2) = td_1 + td_2$ .
5. 最后，当我们什么都没做的时候，每个结点上的标记都是“空的”，表示什么都不做。我们把这个标记记作  $\epsilon$ 。它满足  $\epsilon d = d$ （确实是什么都不做）。
6. 作为一点显然的补充，我们有  $\epsilon t = t\epsilon = t$ 。毕竟  $\epsilon$  就是什么都不做。

数学中，我们把配备了一个二元运算且具有结合律的结构叫做半群。那么可以发现

- 所有信息，加上合并操作，构成了一个半群。
- 所有标记，加上复合操作，构成了一个半群。而且它有一个“单位元”  $\epsilon$ ，所以是么半群（单位元也叫么元，所以么半群其实就是字面意思，带么的半群）。
- 除此之外，标记还会作用在信息上。按数学的话讲，这叫做么半群作用。

这看上去非常的复杂！有没有简单的办法呢？比如说，我懒得想标记和信息是什么形式了，有没有暴力做法呢！

答案是：当然有！虽然很笨蛋！



## “万有”双半群

想象一下，有一个问题，要求区间加和区间乘。但是我偷懒，发现按矩阵的方式，

$$\begin{bmatrix} a & b \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix} = \begin{bmatrix} ax + b \\ 1 \end{bmatrix}.$$

我想起大佬教给我：线段树可以维护矩阵乘法！

于是我每个标记都是一个  $2 \times 2$  的矩阵，信息是  $2 \times 1$  的向量。

可以发现这个例子里，“我”很笨蛋，明明两个数就能维护的标记，我却要用四个数维护。但是虽然笨蛋，却也找到了办法。

有没有“最笨蛋”的办法呢！笨蛋到不需要思考的办法，不需要大佬教矩阵乘法的办法。

答案是确实有。我们考虑这样的抽象的问题：

- 维护一个序列，序列里每个元素是个抽象的数据，具有你不知道具体实现的类型  $M$ 。
- 要求支持操作：给你一个黑盒函数  $M \rightarrow M$ ，以及一个区间，要求你把区间里每个元素都作用上这个函数。
- 支持查询：给你一个黑盒函数  $\text{Array}(M) \rightarrow M$ ，以及一个区间，要求你把这个区间提取出来传到这个函数里，然后递出去结果。这里 Array 是数组，C++ 选手可以理解为 `vector<M>`。

可以发现，所有区间修改区间查询的线段树问题都可以规约到这个问题！（废话.jpg）那么这个问题，怎么使用线段树做呢？（虽然我知道你想说线段树不如暴力）



显然我们没有任何办法压缩标记和信息，毕竟我们对他们一无所知。那么唯一的办法就是： - 一个标记就是一个  $M \rightarrow M$  的函数——不保证这个函数可以  $O(1)$  执行。因为如果传进来一个黑盒函数  $f_1$ , 又传进来黑盒函数  $f_2$ , 那我们只能得到一个新的函数  $f(x) = f_2(f_1(x))$ 。不要在意具体实现——如果你真的在意的话，这可能是一个lambda表达式；你也可以干脆认为一个标记就是一个 vector，里面装着很多黑盒函数，表示要依次执行这些函数。 - 一个信息...就是一个  $\text{Array}(M)$ . 没错，区间维护的信息就是区间本身。因为我们对  $M$  一无所知，完全不能提取出更细致的东西了。很笨蛋！ - 信息的合并...就是俩 Array 拼起来。 - 标记的复合...就是函数复合。 - 标记作用在信息上...就是遍历 Array 里每个元素，扔到函数里拿出来，组成新的 Array。

很笨蛋！由于我们没有任何简化，执行一个函数其实是执行输入的一系列函数的复合，有可能一次就是  $O(n)$  的。所以只是做一次“标记作用在信息上”，就可能  $O(n^2)$  了！最终复杂度可能是  $O(n^3)$ ，不过我们确实是用线段树解决的，不是吗（

为什么要提到这个笨蛋的构造呢？事实上，这个无比憨憨的构造反而是线段树维护区间信息的某种基础，它具有某种“万有性”。

比如说，如果小明和小红分别设计了一套标记+信息的结构，而小明维护的信息以及标记完全可以实现小红的所有信息和标记（也就是说，即使不知道子树长什么样，我们也可以完全通过小明的信息，推算出小红的信息；而且小红的所有标记都可以用小明的标记实现，反过来却不一定），那我们可以认为小明的结构更强。如果这么说，上面这个笨蛋构造就比所有信息+标记结构都强！

但是古尔丹，代价是什么呢？代价当然就是比暴力还要可怜的复杂度。强度也高，复杂度也高，全面吊打

这也是为什么本文把标记的结合叫做复合——它本就是函数的复合。

事实上，这种“万有性”可以启发我们得出以下的事实：

# 瞎编的简单题

- 维护一个序列，支持区间加，求区间平方和

# 瞎编的简单题 \*2

- 维护一个序列，支持区间加，求区间方差



# 瞎编的简单题 \*3

- 维护一个序列，支持单点修改，询问给定一个区间  $l,r$ ，求所有在这个区间  $[l,r]$  中选择两个数相乘所得到的结果之和

# But what is Fenwick tree?

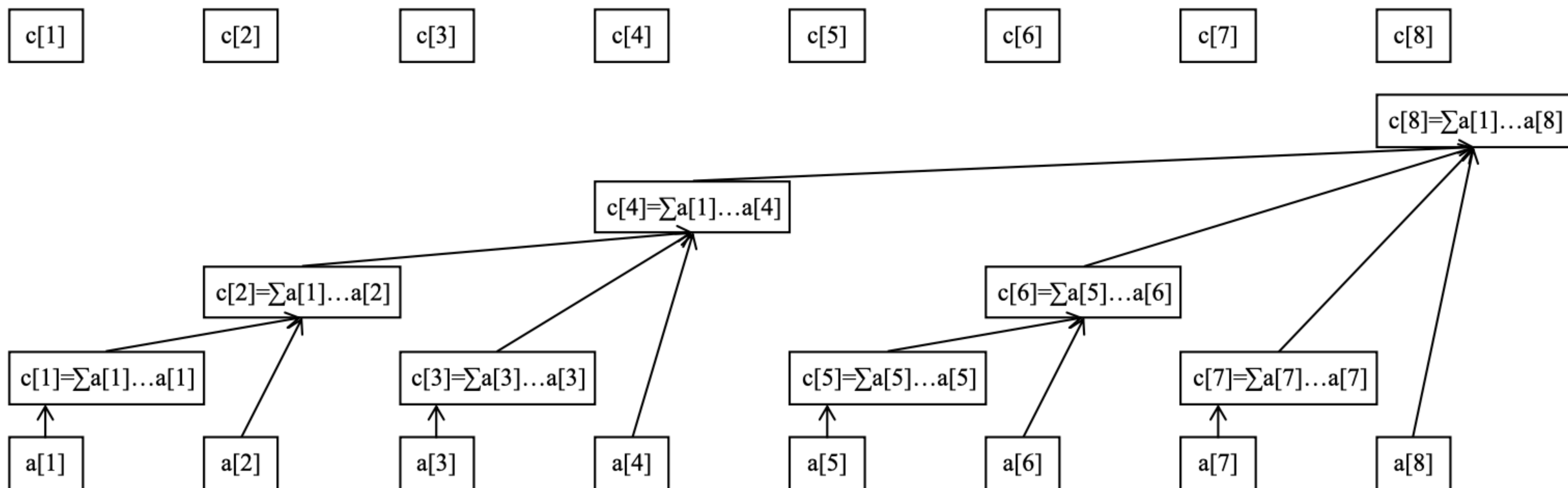
- 树状数组 / Fenwick Tree
- 是什么?

# Idea 是啥?

- 线段树的 idea: 区间分治信息维护
- fenwick 的 idea: ?
- 一组  $[l, r] \Rightarrow O(\log n)$  个节点
- 一段前缀  $[1, r] \Rightarrow O(\log n)$  个节点



# 结构图

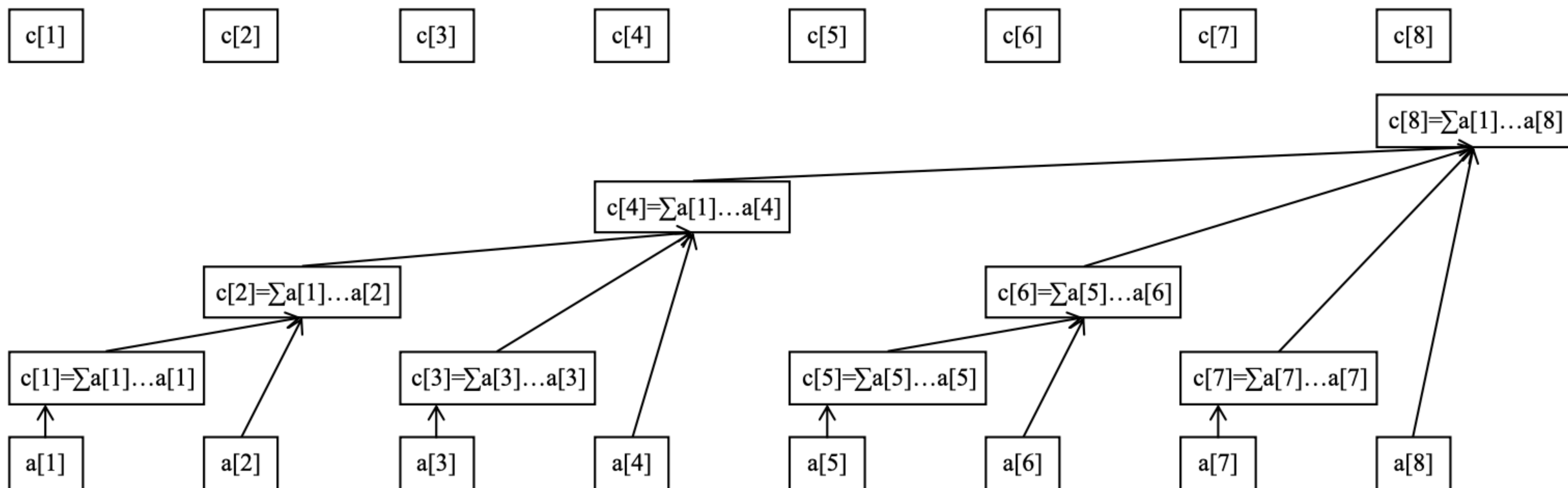


# 本质解释？

- 对于一个  $p$ ，例如  $p=12$
- $p=(1100)_2$
- 取出它的二进制最低非零位
- 对应的是它向上的层



# 结构图



# notes

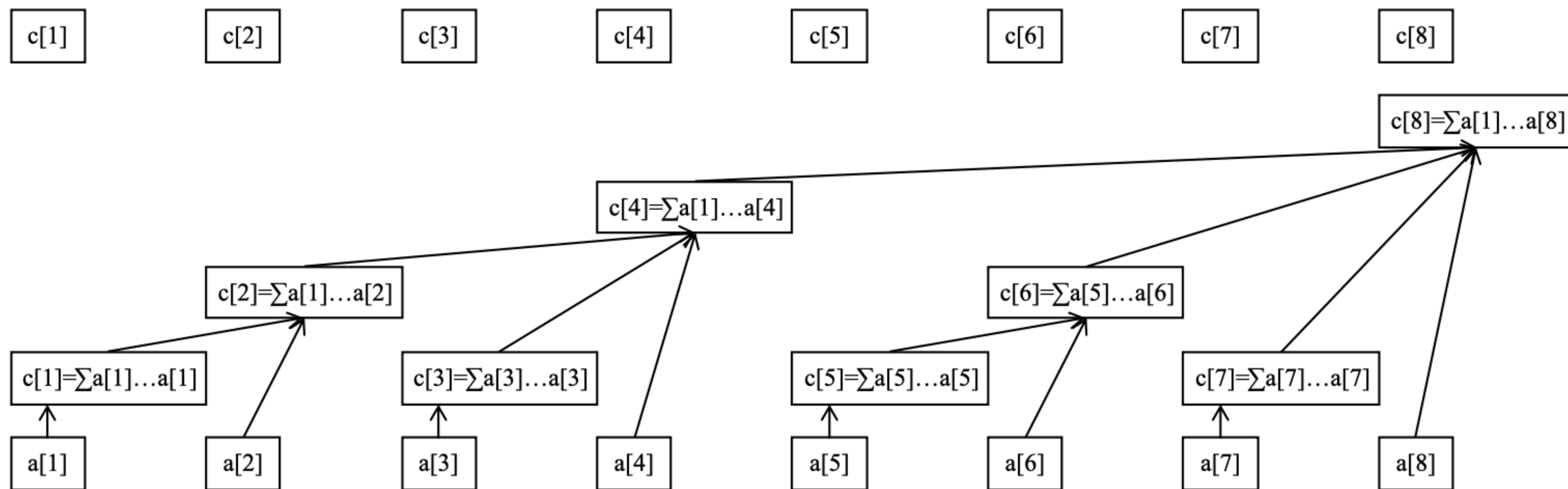
- 优势：好写，常数小
- 问题：对信息的性质要求

# 不可减信息

- 考虑 min、max
- 如果使用树状数组来维护，无法得到区间的信息
- 线段树并不要求信息可减，树状数组则不



# 结构



- 换个角度：线段树删掉所有的右儿子
- 只要信息可减，我们仍然可以差分得到右儿子信息

# 增补

- 如果做区间加，单点求值呢？
  - 相当于是对差分数组单调修改，求前缀和。
- 同样可以扩展到区间加区间求和

# 小练习

- 给定原数组  $a[1 \dots n]$ ，线性时间建出其树状数组。



# Solution

- 每个  $x$  相当于是把自己到根路径全部  $+= x$ ，因为深度只有  $\log$ ，所以复杂度是  $n \log n$
- 建树时，类似树上求前缀和。我们直接更新  $c[x + \text{lowbit}(x)] += c[x]$  即可。

```
//  $\Theta(n)$  建树
void init() {
    for (int i = 1; i <= n; ++i) {
        t[i] += a[i];
        int j = i + lowbit(i);
        if (j <= n) t[j] += t[i];
    }
}
```

# Fenwick Tree

```
def update(pos, val):  
    while (pos <= n):  
        c[pos] += val  
        pos += pos & (-pos)
```

为  $n$  的数组  $c[1\dots n]$ ，在它上面建树状数组

- 你可以随意进行一些操作  $\text{update}(\text{pos}, \text{val})$ ，要求  $1 \leq \text{pos} \leq n$  为整数， $\text{val}$  可以为任意实数
- 告诉你最后  $c$  哪些位置非零，问最少进行多少次  $\text{update}$  操作。  $n \leq 1e6$
- <https://qoj.ac/problem/5075>



# Solution

- 问题就是给你一棵树，每次可以把任意一个点到根的路径  $+= v$ （可以是负数），问最少几次使得给定的点非零，其余点全零。
- 如果一个  $x$  是非零，且有一个儿子也非零，那么在操作儿子的时候顺便就把自己操作了，就不需要额外操作。否则需要额外操作一次。
- 如果一个  $x$  是零，且有儿子非零，那么在修改儿子那一轮必定会影响到自己。
  - 如果有大于等于2个儿子非零，那么容易发现我只需要在另一个儿子处取  $-x$  即可。否则我们必定需要额外一次操作把  $x$  变为零
- 因此答案即为满足自己非零且所有儿子全为零的点的数量 加上 自己为零且恰好有一个儿子非零的点的数量

# 增补： 二维数点

- 给定  $n$  个点，第  $i$  个点的坐标为  $(x_i, y_i)$
- 对每个  $1 \leq i \leq n$  求有多少个点  $j$  满足  $x_i \leq x_j$  且  $y_i \leq y_j$

# Notes

- 许多问题最终可以转化成若干个维度的数点问题
- 例如优化一些 dp，或者计算一些带有约束的和式的贡献



# 扫描线？

- 提到二维数点，顺便提及扫描线。
- 在做二维数点时我们排序第一维，按照从小到大的顺序枚举这个维度
- 这个过程可以形象的被视为扫描线
- <https://oi.wiki/geometry/scanning/>
- （这里有个 gif 图）

# 简单小例题： 矩形面积并

- 给定平面上的若干个矩形，求这些矩形的并的面积
- 一个区域被覆盖多次我们只算一次

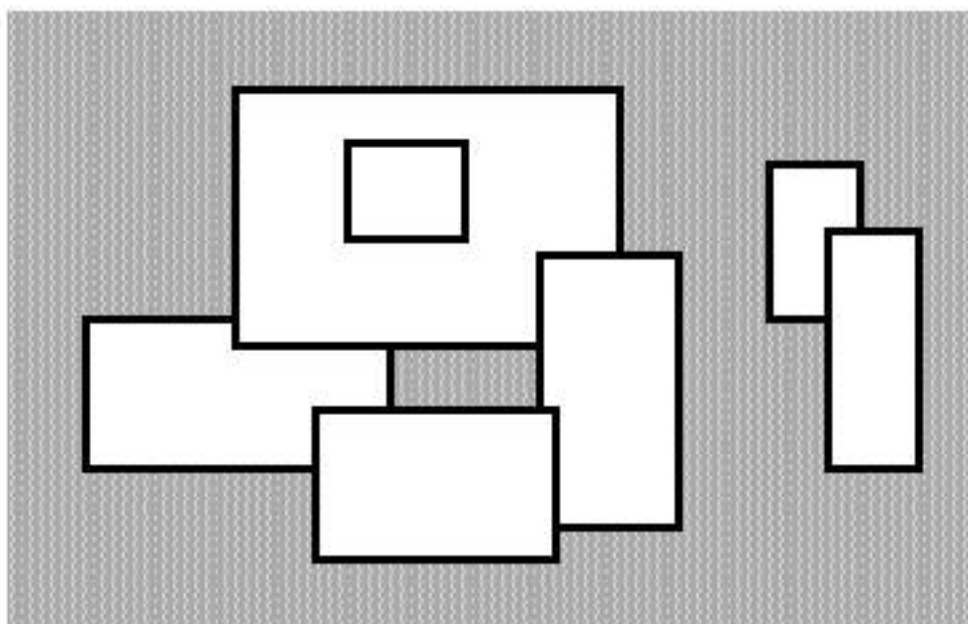


Figure 1. A set of 7 rectangles

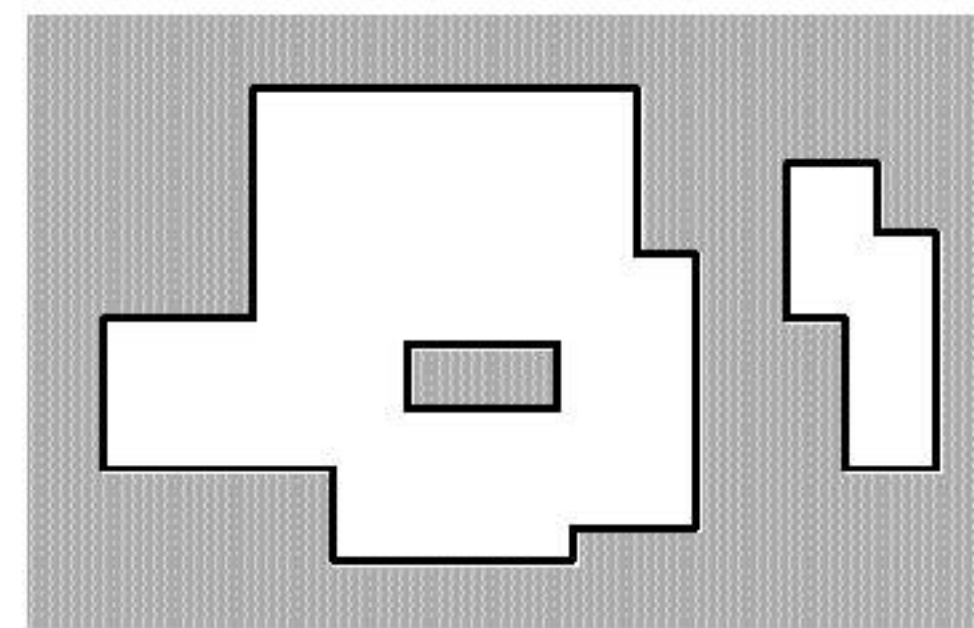


Figure 2. The boundary of the set of rectangles

# 不简单小例题： Captivating process

- 这个题挺难，如果时间不够可能就跳了。
- 有两个  $1-n$  的序列  $f$  和  $g$ 。定义  $f^k[x]$  为执行  $k$  次  $x = f[x]$  后  $x$  得到的值。
- $q$  次询问，每次给定  $x$  和  $y$ ，问是否存在  $k$  使得  $f^k[x] = g^k[y]$
- $1 \leq n, q \leq 10^5$
- <https://qoj.ac/problem/4758>



# 动态开点线段树

- 维护一个长度为  $n$  的序列，初始全0，支持区间加于区间求和。
- $n \leq 1e9$ ,  $q \leq 1e5$

# Idea

- 我们知道一次修改只会定位到  $O(\log n)$  个区间。
- 真的需要一开始将整棵树建出来吗？



# 动态开点

- 只有在需要时才创建节点。
- 这样一次操作我们最多只会增加  $\log(n)$  个节点
- 需要注意的是，在线段树中会有  $O(n)$  个节点，但是如果  $n$  极大而采用动态开点线段树，那么我们消耗的空间是  $O(q \log n)$  的
- 需要注意数组的大小

# 还有啥用？

- 一个更加常见的应用是值域线段树。
- 我们使用线段树来维护一组元素的值。

# 很笨蛋小例题

- 维护一个集合，支持插入一个数 $x$ ，删除一个数 $x$ ，以及询问所有值大于等于 $L$ 且小于等于 $R$ 的元素之和



# 笨蛋小例题

- $n$ 个点，第  $i$  个点有权值  $v[i]$  和颜色  $c[i]$ 。你需要支持以下操作：
  - 给定  $i$   $x$ ，将第  $i$  个点的颜色改为  $x$
  - 给定  $l$   $r$   $k$ ，询问  $l \dots r$  中颜色为  $k$  的所有元素之和

# 线段树二分

- 维护一个正整数序列，支持区间加正整数，以及询问给定一个 $w$ ，求最小的 $r \geq x$ ，使得 $[1 \dots r]$ 的和至少为 $w$

# 线段树+二分 vs 线段树二分

- $r$ 显然具有单调性
- 每次二分，然后线段树询问？
  - $N \log^2 N$
- 直接在线段树上二分！



# 线段树二分 ex

- 维护一个正整数序列，支持区间加正整数，以及询问给定一个 $w$ 和 $x$ ，求最小的 $r \geq x$ ，使得 $[x \dots r]$ 的和至少为 $w$
- （虽然你可以直接把  $[1 \dots x-1]$  的贡献加进去。

# gdpc题

- $n$  个格子排成一行，第 $i$ 个位置有颜色  $c[i]$  和权值  $v[i]$ 。  $q$ 次操作：
- (1) 修改  $v[i] \leftarrow j$ ;
- (2) 修改  $c[i] \leftarrow j$ ;
- (3) 给定起始点  $s$  与合法颜色状态  $c[1], c[2], \dots, c[k]$ ，可以左右走，要求只能走这 $k$ 种颜色的位置，问能走到的格子权值之和
- $n, q \leq 10^5$ ;  $\sum k \leq 10^6$
- <https://uoj.ac/problem/6533>



# Solution

- 只需要找到从s能走的区间  $[l...r]$  的左右端点，相当于找到 s 左侧最后一个没出现过的颜色与右侧第一个没出现过的颜色。
- 怎么找？在这些线段树上一一起线段树二分！
- 找到以后就是一个区间求和。

# 线段树合并

- 小例题
- 给定一棵有根树，每个点有一个颜色。
- $q$  次询问，每次给定  $x$  和  $c$ 。问  $x$  所在的子树中颜色  $c$  出现的次数。
- （虽然你可以转成区间颜色数

# 过程

- 以例题的题解为例。
- 我们使用线段树维护每个点所在子树中每个颜色出现多少次。
- 即，线段树的下标为颜色，而权值则为出现次数。
- 当  $x$  走到  $fa[x]$  时，我们需要快速将  $x$  所在的树合并到  $fa[x]$  上



# 过程

- 怎么合并呢？事实上我们可以暴力合并！
- 具体的，我们同时在被合并的树和合并到的树上进行遍历
- 如果当前被遍历到的点在两棵树上均出现，则我们暴力更新
- 否则，若只在一棵树上出现，则我们可以直接把这棵树挂到被合并的结果上，退出过程

# 复杂度

- 看上去很暴力？
- 但是每进行一次便利，节点总数都会减一！（为什么？）
- 一共只有  $n \log n$  个点，所以总复杂度是  $n \log n$  的！

# 树

- 给定一棵  $n$  个结点的有根树  $T$ ，每个结点有一个正整数权值  $v_i$ 。
- 设  $x$  号结点的子树内的所有结点编号为  $c_1, c_2, \dots, c_k$ ，定义  $x$  的价值为：
$$val(x) = (v_{c_1} + d(c_1, x)) \oplus (v_{c_2} + d(c_2, x)) \oplus \dots \oplus (v_{c_k} + d(c_k, x))$$
- $d(x, y)$  表示树上  $x$  号点与  $y$  号点间唯一简单路径所包含的边数， $\oplus$  表示异或运算。
- 请你求出  $\sum_{i=1}^n val(i)$  的结果。
- <https://qoj.ac/problem/6160>



# Notes

- 一个没有很常见的 trick：从低到高建 trie
- 这样可以方便维护加1操作。
- 由于和今天讲课内容关系不大，具体不做展开。